

```
"""
```

```
The "brain" behind the stumble button.
```

```
Given a user, picks one item they haven't seen yet, weighted toward  
tags they've thumbs-upped before and away from tags they've thumbs-  
downed. New users with no history get pure random -- that's fine,  
personalization only needs to kick in once there's signal.
```

```
"""
```

```
import random
```

```
from db import get_conn, get_tag_scores
```

```
def _item_weight(item_tags: list[str], tag_scores: dict) -> float:
```

```
    """
```

```
    Base weight of 1 for every item (so brand-new users still see  
    everything). Each matching tag nudges the weight up or down based  
    on past reactions. Floored at 0.1 so a disliked tag makes an item  
    rare, not literally impossible -- people's taste does drift.
```

```
    """
```

```
    weight = 1.0
```

```
    for tag in item_tags:
```

```
        weight += tag_scores.get(tag, 0) * 0.5
```

```
    return max(weight, 0.1)
```

```
def get_next_stumble(user_id: int, conn=None) -> dict | None:
```

```
    own_conn = conn is None
```

```
    if own_conn:
```

```
        conn = get_conn().__enter__()
```

```
    try:
```

```
        tag_scores = get_tag_scores(conn, user_id)
```

```
        rows = conn.execute(  
            """SELECT items.* FROM items  
                WHERE is_dead = 0  
                AND id NOT IN (SELECT item_id FROM seen_items WHERE user_id = ?)  
                LIMIT 500""", # cap the candidate pool for performance at scale  
            (user_id,),  
        ).fetchall()
```

```
        """SELECT items.* FROM items
```

```
        WHERE is_dead = 0
```

```
        AND id NOT IN (SELECT item_id FROM seen_items WHERE user_id = ?)
```

```
        LIMIT 500""", # cap the candidate pool for performance at scale
```

```
        (user_id,),
```

```
    ).fetchall()
```

```
    if not rows:
```

```
        return None # user has seen everything -- time to widen the pool or

    candidates = [dict(row) for row in rows]
    weights = [_item_weight(c["tags"].split(","), tag_scores) for c in candid

    chosen = random.choices(candidates, weights=weights, k=1)[0]
    return chosen
finally:
    if own_conn:
        conn.__exit__(None, None, None)
```